PDS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 4 | AULA 07 -

NEW RELIC - HOSTS, CONTAINERS E MÉTRICAS

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SUMÁRIO

O QUE VEM POR AI? 3
HANDS ON 4
SAIBA MAIS 5
O QUE VOCÊ VIU NESTA AULA? 26
REFERÊNCIAS 27

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VEM POR AI?

Esta aula aprofunda o monitoramento de **infraestrutura moderna** utilizando a plataforma **New Relic**, com foco nos pilares de **hosts, containers e métricas**. O conteúdo aborda os desafios de observabilidade em ambientes distribuídos, dinâmicos e efêmeros, característicos de arquiteturas baseadas em containers e orquestradores como Kubernetes.

São apresentados os conceitos fundamentais do **Infrastructure Agent**, sua atuação em nível de sistema operacional e sua adaptação para ambientes conteinerizados. A aula também explora o modelo de dados do New Relic para infraestrutura, destacando como métricas de hosts e containers são coletadas, enriquecidas com metadados e armazenadas no NRDB, permitindo análises históricas, correlação entre camadas e diagnósticos precisos de problemas de desempenho e capacidade.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

HANDS ON

As atividades práticas concentram-se na instalação, configuração e exploração do New Relic Infrastructure Agent em diferentes contextos de execução, incluindo hosts tradicionais e ambientes baseados em containers.

O hands-on contempla a coleta e análise de métricas de CPU, memória, disco, rede e processos, além da observação do comportamento de contêineres individuais por meio do evento ContainerSample. São realizadas consultas NRQL para agregação, filtragem e visualização de métricas, bem como a construção de dashboards básicos e a definição de condições de alerta baseadas em métricas de infraestrutura.

As práticas demonstram como a observabilidade de infraestrutura evolui de comandos locais e efêmeros para uma plataforma centralizada, persistente e orientada a dados, capaz de sustentar análises em escala e apoiar processos de resposta a incidentes.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SAIBA MAIS

Instalação De Agente No Host

Infra Agent como Coletor Os-Level

O New Relic Infrastructure Agent é um daemon leve e de alta performance projetado para ser executado em seus hosts (servidores físicos, máquinas virtuais, instâncias na nuvem) com o objetivo principal de coletar dados em nível de sistema operacional (OS-level). Ele atua como um sensor local, coletando uma vasta gama de métricas de telemetria sobre a saúde e o desempenho do sistema. Essas métricas incluem (mas não se limitam) uso de **CPU**, consumo de **memória**, utilização de **disco** (I/O e capacidade), tráfego de **rede** e informações detalhadas sobre **processos** em execução.

Uma vez coletados, esses dados são empacotados e enviados de forma segura para a plataforma New Relic, onde são processados, armazenados no New Relic Database (NRDB) e disponibilizados para visualização, consulta e alerta. A arquitetura do agent foi pensada para ter um baixo overhead, garantindo que o monitoramento não impacte negativamente a performance das aplicações que estão sendo observadas.

O Infrastructure Agent é a fundação sobre a qual outras formas de monitoramento de infraestrutura são construídas. Ele não apenas fornece uma visão completa do host, mas também serve como um ponto de coleta para integrações on-host, que estendem sua capacidade para monitorar serviços específicos como bancos de dados (MySQL, PostgreSQL), web servers (NGINX, Apache) e muito mais. Além disso, ele é a peça-chave para o monitoramento de containers, como sendo capaz de detectar e coletar métricas de ambientes Docker e Kubernetes.

O Infrastructure Agent é um componente fundamental da plataforma New Relic, responsável por coletar, processar e reportar dados sobre a saúde e a performance de seus hosts. Ele fornece a telemetria básica necessária para entender o comportamento de sua infraestrutura e como ela suporta suas aplicações.

Compreender o papel do Infrastructure Agent como um coletor OS-level é o primeiro passo para construir uma estratégia de observabilidade robusta, permitindo

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

que equipes de DevOps e SRE identifiquem proativamente gargalos de performance, otimizem o uso de recursos e resolvam incidentes com maior rapidez e eficiência.

Categoria de Métrica	Exemplos de Dados Coletados
CPU	Percentual de uso (user, system, iowait), carga do sistema (load average), número de cores
Memória	Memória total, usada, livre, em cache, swap
Disco	IOPS (operações de leitura/escrita por segundo), throughput (bytes/segundo), espaço total/usado/livre por partição
Rede	Pacotes/bytes enviados e recebidos, pacotes com erro, informações de interface de rede
Processos	Uso de CPU e memória por processo, PID, usuário, linha de comando

Figura 1 – Infrastructure Agent
Fonte: Elaborado pelo autor (2026)

Instalação e Configuração

A instalação do New Relic Infrastructure Agent é um processo direto e bem documentado, utilizando os gerenciadores de pacotes nativos dos principais sistemas operacionais Linux, como APT para Debian/Ubuntu e YUM para RHEL/CentOS. Este método é o preferido, pois facilita não apenas a instalação inicial, mas também futuras atualizações e a manutenção do agent.

O processo de instalação consiste em algumas etapas-chave:

- Adicionar o repositório do New Relic: para que o gerenciador de pacotes possa encontrar o newrelic-infra, é necessário primeiro adicionar o repositório oficial da New Relic à lista de fontes de software do sistema.
- Importar a chave GPG: para garantir a autenticidade dos pacotes, a chave GPG da New Relic deve ser importada.
- Instalar o pacote: com o repositório configurado, a instalação é feita com um simples comando (sudo apt-get install newrelic-infra ou sudo yum install newrelic-infra).

Após a instalação, a configuração principal do agent é realizada através do arquivo newrelic-infra.yml, localizado em /etc/. A única configuração estritamente

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

obrigatória é a license_key, que autentica o agent e o associa à sua conta New Relic. No entanto, o arquivo de configuração oferece uma vasta gama de opções para customizar o comportamento do agent, como:

- display_name: define um nome de exibição customizado para o host na UI do New Relic, útil para identificação.
- custom_attributes: permite adicionar metadados customizados (tags) ao host, como ambiente: producao ou equipe: backend. Esses atributos são fundamentais para filtrar, agrupar e consultar seus dados.
- log_file: especifica um local alternativo para os logs do próprio agent.
- Configurações de proxy: permite que o agent se comunique com a New Relic através de um proxy HTTP.

É uma boa prática gerenciar o arquivo newrelic-infra.yml utilizando ferramentas de automação e gerenciamento de configuração como Ansible, Puppet ou Chef, garantindo consistência em toda a sua frota de servidores.

Instalação e Configuração

É uma boa prática gerenciar o arquivo newrelic-infra.yml utilizando ferramentas de automação e gerenciamento de configuração como Ansible, Puppet ou Chef, garantindo consistência em toda a sua frota de servidores.

Nesta seção, vamos instalar e configurar o New Relic Infrastructure Agent em uma máquina virtual Ubuntu 22.04. Em seguida, criaremos uma aplicação .NET Core simples para gerar métricas de processo que poderão ser observadas no New Relic.

Pré-requisitos:

Uma máquina virtual (ou host físico) com Ubuntu 22.04.

Acesso root ou sudo.

Uma conta New Relic e sua License Key. Você pode obtê-la em one.newrelic.com > (User Menu) > API keys.

SDK do .NET 6 ou superior instalado no host.

Passo 1: Instalar o .NET SDK

Se você ainda não tem o .NET SDK instalado, execute os seguintes comandos:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

```
sudo apt-get update && \
     sudo apt-get install -y dotnet-sdk-6.0
```

Passo 2: Instalar o Infrastructure Agent

Siga os comandos a seguir para adicionar o repositório do New Relic e instalar o pacote newrelic-infra.

```
# Adicionar a chave GPG do New Relic
curl -s
https://download.newrelic.com/infrastructure_agent/gpg/newreli
c-infra.gpg | sudo apt-key add -

# Adicionar o repositório do New Relic
printf "deb [arch=amd64]
https://download.newrelic.com/infrastructure_agent/linux/apt
focal main" | sudo tee -a /etc/apt/sources.list.d/newrelic-
infra.list

# Atualizar o gerenciador de pacotes e instalar o agent
sudo apt-get update
sudo apt-get install newrelic-infra -y
```

Passo 3: Configurar o Agent

Agora, vamos editar o arquivo de configuração para adicionar sua licença e um nome de exibição para o host.

- Abra o arquivo de configuração em um editor de texto:

sudo nano /etc/newrelic-infra.yml

- Adicione sua license_key e um display_name ao arquivo. Você pode adicionar o display_name em qualquer lugar do arquivo.

```
license_key: SUA_LICENSE_KEY_AQUI
     display_name: fiap-apm-host-01
```

- Salve e feche o arquivo (em nano, use Ctrl+X, depois Y e Enter).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Passo 4: Iniciar o Agent e Verificar o Status

Inicie o serviço do agent e verifique se ele está rodando corretamente.

```
# Iniciar o serviço do agent
     sudo systemctl start newrelic-infra

     # Verificar o status do serviço
     sudo systemctl status newrelic-infra
```

A saída deve mostrar o serviço como active (running). Em poucos minutos, você deverá ver seu host fiap-apm-host-01 aparecendo na UI do New Relic em Infrastructure > Hosts.

Passo 5: Criar e Rodar uma Aplicação .NET para Gerar Carga

Para que o agent tenha processos para monitorar, vamos criar uma simples aplicação de console .NET que consome um pouco de CPU.

- Crie um novo projeto de console:

```
dotnet new console -o Fiap.Apm.CpuLoader
     cd Fiap.Apm.CpuLoader
```

- Substitua o conteúdo do arquivo Program.cs pelo código a seguir. Este código inicia um loop infinito que realiza um cálculo, mantendo a CPU ocupada.

```
     Console.WriteLine("Iniciando aplicação para gerar carga de
CPU...");
     Console.WriteLine("Pressione Ctrl+C para sair.");

     // Loop infinito para consumir CPU
     while (true)
     {
          // Realiza um cálculo simples para manter a CPU ocupada
          double result = 0;
          for (int i = 0; i < 1000000; i++)
          {
               result += Math.Sqrt(i);
          }
     }
```

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Código-fonte 1 – C#
Fonte: Elaborado pelo autor (2026)

- Execute a aplicação:

```
dotnet run
```

Deixe a aplicação rodando. Agora, volte para a UI do New Relic. Na página do seu host, vá para a aba “Processes”. Você poderá filtrar pelo nome do processo dotnet e observar o consumo de CPU e memória da sua aplicação em tempo real. Isso demonstra como o Infrastructure Agent captura métricas de processos específicos, permitindo a análise detalhada do comportamento de suas aplicações no nível do host.

Inventário e Metadata Automática

Além das métricas de performance (como uso de CPU e memória), um dos recursos mais poderosos do Infrastructure Agent é a coleta automática de dados de inventário e metadados do host. Esses dados fornecem um snapshot detalhado da configuração do sistema, que é crucial para o gerenciamento de configurações, auditoria de segurança e troubleshooting de incidentes.

Ao contrário das métricas de série temporal, que são coletadas a cada poucos segundos, os dados de inventário são coletados em intervalos maiores e apenas quando ocorrem mudanças. Isso cria um registro histórico das alterações de configuração do host. Quando uma mudança é detectada (por exemplo, um pacote de software é instalado ou um serviço é iniciado), o agent gera um evento do tipo InfrastructureEvent, que é registrado no NRDB.

Os dados de inventário coletados incluem:

- Pacotes de Software: lista de todos os pacotes instalados no sistema, incluindo seus nomes e versões (ex: openssh-server 7.6p1).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- Serviços do Sistema: status de todos os serviços gerenciados pelo systemd ou outro init system (ex: sshd, nginx).
- Configurações do Kernel: parâmetros de configuração do kernel do Linux.
- Informações do Sistema: versão do sistema operacional, hostname, tipo de instância (em ambientes de nuvem como AWS EC2), e muito mais.

Esses metadados são automaticamente associados a todas as métricas coletadas pelo agent. Por exemplo, quando você visualiza o uso de CPU de um host, você também pode ver a versão do kernel, o tipo de instância EC2 e todos os outros metadados coletados. Essa contextualização é imensamente valiosa. Imagine poder responder a perguntas como:

- "Mostre-me o uso médio de CPU em todos os hosts com a versão 1.20.1 do NGINX."
- "Quais hosts no nosso ambiente de produção ainda estão executando uma versão vulnerável do OpenSSL?"
- "Houve alguma alteração de configuração no host webapp-03 pouco antes do pico de latência que ocorreu às 15:00?"

Essa capacidade de correlacionar dados de performance com dados de configuração transforma o New Relic de uma simples ferramenta de monitoramento em uma poderosa plataforma de observabilidade e análise de infraestrutura.

Inventário e Metadata Automática

Vamos explorar os dados de inventário e os metadados que o Infrastructure Agent coletou automaticamente do nosso host. Usaremos a UI do New Relic e a linguagem de consulta NRQL (New Relic Query Language) para inspecionar esses dados.

Pré-requisitos:

O Infrastructure Agent deve estar instalado, configurado e reportando dados para o New Relic, conforme o passo a passo anterior.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Passo 1: Adicionar Atributos Customizados

Metadados são mais poderosos quando customizados para o seu contexto. Vamos adicionar alguns atributos customizados ao nosso host para simular um ambiente real, onde hosts pertencem a diferentes ambientes, equipes ou projetos.

- Edite novamente o arquivo de configuração do agent:

```
sudo nano /etc/newrelic-infra.yml
```

- Adicione a seção custom_attributes ao final do arquivo. Esta seção permite definir pares de chave-valor que serão anexados a todos os dados reportados por este host.

```
    # ... (sua license_key e display_name existentes)

custom_attributes:
  ambiente: "pos-graduacao-fiap"
  disciplina: "apm-hands-on"
  turma: "2026-T1"
  projeto: "material-didatico"
```

Código fonte 2 - YAML.
Fonte: Elaborado pelo autor (2026)

- Salve o arquivo e reinicie o serviço do agent para que as novas configurações sejam aplicadas:

```
sudo systemctl restart newrelic-infra
```

Passo 2: Explorar o Inventário na UI do New Relic

- Navegue até a UI do New Relic: one.newrelic.com.
- No menu, vá para Infrastructure > Hosts e selecione o seu host (fiap-apm-host-01).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- No painel de navegação esquerdo da página do host, clique em Inventory. Aqui, você encontrará uma lista de todas as categorias de inventário que o agent coletou. Explore algumas delas:
  - **system**: mostra detalhes como a versão do sistema operacional (linuxDistribution, kernelVersion).
  - **config/ssh**: exibe a configuração do serviço SSH.
  - **packages**: apresenta uma lista completa de todos os pacotes de software instalados. Tente pesquisar por dotnet para ver os pacotes do .NET SDK que instalamos.
- Observe que os atributos customizados que adicionamos (ambiente, disciplina, etc.) agora aparecem na seção de metadados do host, geralmente no topo da página ou em uma aba de "Metadata".

Passo 3: Consultar Metadados com NRQL

A verdadeira força dos metadados está na capacidade de usá-los para consultar e agregar dados de performance. Vamos usar o NRQL para fazer algumas perguntas à nossa infraestrutura.

- Vá para one.newrelic.com > Query your data.
- Execute a seguinte consulta. Ela retorna o uso médio de CPU (cpuPercent) para todos os hosts, mas agrupa (FACET) os resultados pelo nosso atributo customizado ambiente.

```
SELECT average(cpuPercent) FROM SystemSample FACET ambiente
```

Como só temos um host, veremos apenas um resultado. Mas em um ambiente com centenas de hosts, essa consulta separaria o uso de CPU por cada ambiente definido (desenvolvimento, homologação, produção, etc.).

- Agora, vamos consultar o inventário. A consulta a seguir nos permite encontrar todos os hosts que possuem o pacote dotnet-sdk-6.0 instalado.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

```
SELECT uniques(hostname) FROM SystemSample WHERE
apmApplicationNames LIKE '%Fiap.Apm.CpuLoader%'
```

Nota: A consulta direta de dados de inventário com NRQL pode ser complexa. Uma abordagem mais comum é usar os atributos de inventário para filtrar eventos de métricas, como SystemSample.

- Vamos tentar uma consulta que correlaciona um evento de processo com um metadado do host. Esta consulta encontra o consumo de memória para o processo dotnet que está rodando no ambiente pos-graduacao-fiap.

```
SELECT average(memoryResidentSizeBytes) FROM ProcessSample
WHERE processDisplayName = 'dotnet' AND ambiente = 'pos-
graduacao-fiap' FACET hostname
```

Esses exemplos demonstram como a coleta automática de inventário e a capacidade de adicionar metadados customizados são fundamentais para a observabilidade. Eles permitem não apenas ver o que está acontecendo (métricas de performance), mas também entender o contexto de onde e em que configuração isso está acontecendo, acelerando drasticamente a análise de causa raiz e o planejamento de capacidade.

Instalando o Agent Em Containers

Auto-Discovery do Runtime Docker

O monitoramento de contêineres apresenta desafios únicos em comparação com hosts tradicionais. Contêineres são efêmeros, com ciclos de vida curtos, e podem ser criados e destruídos em grande escala e alta velocidade. Uma abordagem de monitoramento que exige configuração manual para cada contêiner é, portanto, inviável.

Ciente desse desafio, o New Relic Infrastructure Agent foi projetado com um recurso de descoberta automática (auto-discovery) para ambientes de contêineres.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Quando o agent está em execução em um host que possui um runtime de contêiner compatível, como o Docker, ele detecta automaticamente a presença do runtime e começa a monitorar todos os contêineres em execução nesse host, sem a necessidade de qualquer configuração adicional.

O mecanismo de descoberta funciona da seguinte forma:

Detecção do Socket: o agent procura pela presença do socket do Docker, que por padrão está localizado em /var/run/docker.sock. Este socket é a API através da qual o cliente Docker (e outras ferramentas, como o agent do New Relic) se comunica com o daemon do Docker.

Coleta de Metadados: uma vez conectado ao daemon, o agent utiliza a API do Docker para obter uma lista de todos os contêineres em execução, bem como metadados detalhados sobre cada um, como ID do contêiner, nome, imagem utilizada, labels e portas mapeadas.

Coleta de Métricas: o agent então começa a coletar métricas de performance para cada contêiner descoberto. Ele utiliza os mecanismos do próprio kernel Linux (cgroups) para medir com precisão o consumo de recursos (CPU, memória, I/O de rede e disco) de cada contêiner isoladamente.

Geração de Eventos ContainerSample: os dados coletados são então agregados e enviados para a plataforma New Relic como eventos do tipo ContainerSample. Cada evento ContainerSample representa um snapshot das métricas e metadados de um contêiner específico em um determinado momento. Esses eventos são gerados a cada 15 segundos por padrão.

Essa abordagem de descoberta automática simplifica drasticamente o monitoramento de contêineres. Os desenvolvedores e operadores não precisam se preocupar em instrumentar cada contêiner individualmente. Basta garantir que o Infrastructure Agent esteja instalado no host subjacente, e o New Relic cuidará do resto, fornecendo visibilidade imediata sobre a saúde e o desempenho de todo o ambiente de contêineres.

A capacidade de auto-discovery do New Relic Infrastructure Agent elimina a complexidade da configuração de monitoramento em ambientes Docker. Ao detectar e instrumentar automaticamente todos os contêineres em um host, ele fornece visibilidade instantânea e completa, permitindo que as equipes se concentrem no

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

desenvolvimento e na operação de suas aplicações, em vez de na configuração de ferramentas de monitoramento.

Essa funcionalidade é a base para o monitoramento de contêineres no New Relic e se estende a orquestradores mais complexos, como será visto na seção sobre Kubernetes.

Auto-Discovery do Runtime Docker

Vamos testemunhar o recurso de auto-discovery em ação. Iremos containerizar uma aplicação web ASP.NET Core usando Docker e observar como o Infrastructure Agent, que já está rodando no nosso host, a detecta e começa a monitorá-la automaticamente.

Pré-requisitos:

- O Infrastructure Agent instalado e rodando no host.
- Docker instalado no host. Se não estiver instalado, execute: sudo apt-get install docker.io -y e depois sudo systemctl start docker e sudo systemctl enable docker.
- SDK do .NET 6 ou superior.

Passo 1: Criar uma Aplicação Web ASP.NET Core

Primeiro, vamos criar uma aplicação web simples que servirá como nosso alvo para containerização.

Crie um novo projeto de Web API:

```
dotnet new webapi -o Fiap.Apm.ContainerApi
     cd Fiap.Apm.ContainerApi
```

Para este exemplo, a API padrão criada pelo template é suficiente. Ela já expõe alguns endpoints, como /weatherforecast.

Passo 2: Criar o Dockerfile

O Dockerfile é um script que contém as instruções para montar a imagem do nosso contêiner. Crie um arquivo chamado Dockerfile (sem extensão) dentro do diretório Fiap.Apm.ContainerApi.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

```
nano Dockerfile
```

Cole o seguinte conteúdo no arquivo. Este é um Dockerfile multi-stage, uma prática recomendada para criar imagens .NET otimizadas.

```
# Stage 1: Build
FROM mcr.microsoft.com/dotnet/sdk:6.0 AS build
WORKDIR /app

# Copia os arquivos de projeto e restaura as dependências
COPY *.csproj .
RUN dotnet restore

# Copia o resto do código e publica a aplicação
COPY . .
RUN dotnet publish -c Release -o out

# Stage 2: Runtime
FROM mcr.microsoft.com/dotnet/aspnet:6.0 AS runtime
WORKDIR /app
COPY --from=build /app/out .
ENTRYPOINT ["dotnet", "Fiap.Apm.ContainerApi.dll"]
```

Passo 3: Construir a Imagem Docker e Executar o Contêiner

Com o Dockerfile criado, vamos construir a imagem. O comando a seguir constrói a imagem e a nomeia (tag) como fiap-apm-api.

```
sudo docker build -t fiap-apm-api .
```

Após a construção da imagem, vamos executar um contêiner a partir dela. Vamos mapear a porta 80 do contêiner para a porta 8080 do host.

```
sudo docker run -d -p 8080:80 --name fiap-api-container fiap-
apm-api
```

- -d: executa o contêiner em modo detached (em segundo plano).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- -p 8080:80: mapeia a porta 8080 do host para a porta 80 do contêiner.
- --name fiap-api-container: dá um nome amigável ao nosso contêiner.

Verifique se o contêiner está em execução:

sudo docker ps

Você deverá ver fiap-api-container na lista.

Passo 4: Verificar o Auto-Discovery na UI do New Relic

O Infrastructure Agent no host detectará o novo contêiner em poucos segundos.

- Vá para a UI do New Relic: one.newrelic.com.
- Navegue para Infrastructure > Hosts e selecione o seu host (fiap-apm-host-01).
- Clique na aba Containers.

Você verá o fiap-api-container listado ali. O New Relic já está coletando métricas de CPU, memória e rede para ele. Clique no nome do contêiner para ver um dashboard detalhado com todas as métricas e metadados associados, como a imagem (fiap-apm-api) e o ID do contêiner.

Passo 5: Consultar Dados do Contêiner com NRQL

Vamos usar o NRQL para consultar os dados do ContainerSample que estão sendo gerados.

- Vá para one.newrelic.com > Query your data.
- Execute a seguinte consulta para ver o número de contêineres por imagem:

```
SELECT uniqueCount(containerId) FROM ContainerSample FACET
imageName
```

- Agora, vamos ver o uso de memória do nosso contêiner específico ao longo do tempo:

```
SELECT average(memoryUsageBytes) FROM ContainerSample WHERE
containerName = 'fiap-api-container' TIMESERIES
```

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Este exercício prático demonstra a simplicidade e o poder do auto-discovery. Sem nenhuma configuração no New Relic ou na nossa aplicação, obtivemos visibilidade completa sobre um contêiner recém-lançado, simplesmente por ter o Infrastructure Agent rodando no host. Isso mostra como a plataforma foi construída para se adaptar perfeitamente a fluxos de trabalho modernos de CI/CD e ambientes dinâmicos baseados em contêineres.

Infra Agent Rodando como Container

Embora a instalação do Infrastructure Agent diretamente no sistema operacional do host seja o método mais comum, existem cenários onde essa abordagem não é ideal ou possível. Ambientes que utilizam sistemas operacionais otimizados para contêineres (como RancherOS, CoreOS ou Bottlerocket da AWS) ou políticas de segurança que restringem a instalação de software no host, exigem uma abordagem diferente. Para atender a esses casos, a New Relic oferece uma imagem Docker oficial do Infrastructure Agent, permitindo que o próprio agent seja executado como um contêiner [8].

O modelo consiste em executar o contêiner do agent em um modo privilegiado, dando a ele acesso a informações e namespaces do host. Isso permite que o agent, mesmo estando isolado em um contêiner, consiga coletar métricas sobre o host subjacente e sobre todos os outros contêineres que estão sendo executados nesse mesmo host. Essencialmente, o agent "enxerga para fora" de seu próprio contêiner.

Para que isso funcione, o contêiner do agent precisa ser iniciado com uma série de flags específicas no comando docker run, que quebram o isolamento padrão dos contêineres de forma controlada:

- --privileged ou --cap-add=SYS_PTRACE: concede ao contêiner permissões elevadas no host, necessárias para inspecionar processos e coletar certos tipos de métricas.
- --pid=host: faz com que o contêiner compartilhe o namespace de PID (Process ID) do host. Isso é fundamental para que o agent possa ver e listar todos os processos que estão rodando no host, e não apenas o seu próprio processo.
- --network=host: conecta o contêiner diretamente à pilha de rede do host, permitindo a coleta precisa de métricas de rede.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- -v "/:/host:ro": monta o sistema de arquivos raiz (/) do host como um volume somente leitura (ro) dentro do contêiner, no diretório /host. Isso dá ao agent acesso a arquivos de sistema (como os do /proc e /sys) para coletar métricas de disco, memória e sistema.
- -v "/var/run/docker.sock:/var/run/docker.sock": monta o socket do Docker do host dentro do contêiner. Esta é a mesma técnica usada para o auto-discovery, permitindo que o agent containerizado se comunique com o daemon do Docker para listar e inspecionar outros contêineres.

Executar o agent como um contêiner oferece uma maneira flexível e declarativa de gerenciar o monitoramento, alinhada com os princípios de infraestrutura como código. A configuração do agent pode ser gerenciada através de variáveis de ambiente no comando docker run ou, de forma mais robusta, construindo uma imagem customizada que herda da imagem oficial e inclui um arquivo newrelic-infra.yml personalizado.

Infra Agent Rodando como Container

Aqui, vamos substituir a instalação do agent baseada em host pela abordagem de agent containerizado. Isso nos permitirá entender na prática como ele funciona e validar que a visibilidade sobre o host e os outros contêineres é mantida.

Pré-requisitos:

- Docker instalado e em execução.
- O contêiner fiap-api-container da seção anterior ainda deve estar em execução para servir como alvo de monitoramento.
- Sua License Key do New Relic.

Passo 1: Parar e Desabilitar o Agent Instalado no Host

Para evitar conflitos, pois só podemos ter uma instância do agent monitorando um host, precisamos primeiro parar e desabilitar o serviço newrelic-infra que instalamos anteriormente.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

```
# Parar o serviço do agent
sudo systemctl stop newrelic-infra

# Desabilitar o serviço para que não inicie com o boot
sudo systemctl disable newrelic-infra

# Opcional: Você pode remover completamente o pacote se
desejar
# sudo apt-get remove newrelic-infra -y
```

Após alguns minutos, o New Relic marcará o host fiap-apm-host-01 como indisponível, o que é esperado.

Passo 2: Executar o Infrastructure Agent como um Contêiner Docker

Agora, vamos iniciar o agent usando a imagem oficial da New Relic. Usaremos o comando docker run com as flags privilegiadas que discutimos na parte teórica. Substitua SUA_LICENSE_KEY_AQUI pela sua chave de licença real.

```
sudo docker run \
    -d \
    --name newrelic-infra \
    --network=host \
    --cap-add=SYS_PTRACE \
    --privileged \
    --pid=host \
    --cgroupns=host \
    -v "/:/host:ro" \
    -v "/var/run/docker.sock:/var/run/docker.sock" \
    -e NRIA_LICENSE_KEY=SUA_LICENSE_KEY_AQUI \
    -e NRIA_DISPLAY_NAME=fiap-apm-host-containerized \
    newrelic/infrastructure:latest
```

Análise do Comando:

- -d: executa em modo detached.
- --name newrelic-infra: nomeia o contêiner do agent.
- As flags --network, --cap-add, --privileged, --pid, --cgroupns e os volumes (-v) concedem as permissões necessárias para o agent monitorar o host.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- -e NRIA_LICENSE_KEY=...: passa a license key como uma variável de ambiente.
- -e NRIA_DISPLAY_NAME=...: define um novo nome de exibição para o host, fiap-apm-host-containerized, para que possamos diferenciá-lo facilmente na UI do New Relic.
- newrelic/infrastructure:latest: especifica a imagem Docker a ser usada.

Passo 3: Verificar a Execução e os Dados na UI

Verifique se o contêiner do agent está em execução:

sudo docker ps

Você deverá ver dois contêineres em execução: newrelic-infra e fiap-api-container.

Acesse a UI do New Relic em Infrastructure > Hosts.

- Você verá um novo host chamado fiap-apm-host-containerized aparecer na lista.
- O host antigo (fiap-apm-host-01) eventualmente ficará cinza e poderá ser removido.

Clique no novo host fiap-apm-host-containerized.

- Observe que todas as métricas do host (CPU, memória, etc.) estão sendo coletadas normalmente, mesmo com o agent rodando dentro de um contêiner.

Vá para a aba Containers dentro da página do host.

- Você verá que o agent containerizado também descobriu e está monitorando o fiap-api-container com sucesso. A visibilidade sobre os outros contêineres no host foi mantida.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Métricas Coletadas e Modelo de Dados

Métricas de Host (Systemsample)

O evento SystemSample é a pedra angular do monitoramento de hosts no New Relic. Ele representa um snapshot abrangente do estado de saúde e performance de um sistema operacional em um determinado momento. Por padrão, o Infrastructure Agent coleta esses dados a cada 5 segundos, fornecendo uma visão de alta granularidade sobre o comportamento do host.

Cada SystemSample é um evento rico em dados, contendo dezenas de atributos que descrevem o uso de recursos fundamentais. Esses atributos podem ser agrupados em quatro categorias principais:

CPU: métricas que detalham a utilização do processador. Isso inclui o cpuPercent (percentual total de uso), que é decomposto em cpuUserPercent, cpuSystemPercent, cpuIoWaitPercent e cpuStealPercent. Essa decomposição é vital para diagnosticar a natureza da carga de CPU. Por exemplo, um cpuIoWaitPercent alto sugere que a CPU está ociosa, aguardando operações de disco, indicando um gargalo de I/O, e não de processamento.

Memória: atributos que descrevem o consumo de memória RAM e de troca (swap). Inclui memoryTotalBytes, memoryUsedBytes, memoryFreeBytes e memoryCachedBytes. A análise dessas métricas ajuda a identificar vazamentos de memória (memory leaks) ou a necessidade de provisionar mais RAM para um host.

Disco: métricas relacionadas ao sistema de armazenamento. O SystemSample captura o diskUsedPercent e diskFreeBytes para dar uma visão geral do espaço em disco. Métricas mais detalhadas sobre performance de I/O (leitura/escrita por segundo) são capturadas em um evento separado, o StorageSample.

Rede: métricas sobre o tráfego de rede. Inclui receiveBytesPerSecond e transmitBytesPerSecond, que medem o throughput de entrada e saída de dados do host. Assim como no disco, métricas mais detalhadas por interface de rede são encontradas no evento NetworkSample.

Além dessas métricas de performance, cada evento SystemSample é enriquecido com todos os metadados do host, incluindo os atributos padrão coletados

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

pelo agent (como hostname, kernelVersion, awsRegion, ec2InstanceType) e quaisquer atributos customizados definidos no arquivo newrelic-infra.yml. Essa combinação de métricas de alta frequência com metadados ricos é o que permite a análise profunda e contextualizada na plataforma.

O SystemSample fornece uma visão holística e regular do estado de um host. Ao consultar esses eventos no NRDB, os engenheiros podem rastrear tendências de performance, correlacionar o uso de recursos com o comportamento da aplicação e diagnosticar problemas de infraestrutura com base em dados empíricos detalhados.

Compreender a estrutura e o conteúdo do SystemSample é o primeiro passo para dominar a consulta de dados de infraestrutura no New Relic e para a criação de dashboards e alertas significativos.

Métricas de Containers (Containersample)

Assim como o SystemSample é a base para o monitoramento de hosts, o evento ContainerSample é o pilar do monitoramento de contêineres no New Relic. Este evento é gerado pelo Infrastructure Agent sempre que ele detecta um ambiente de contêineres (como Docker) em um host monitorado. Por padrão, um ContainerSample é capturado para cada contêiner em execução a cada 15 segundos.

O ContainerSample encapsula métricas de performance e metadados de um único contêiner, fornecendo uma visão detalhada de como cada contêiner está utilizando os recursos alocados a ele pelo host. A estrutura deste evento é projetada para responder a perguntas críticas em ambientes de microsserviços, como: "Qual contêiner está consumindo mais CPU?", "A minha aplicação está sofrendo com limitação de memória (throttling)?" ou "Qual imagem de contêiner está associada a um pico de tráfego de rede?".

Os principais atributos do ContainerSample podem ser categorizados da seguinte forma:

Identificação e Metadados: atributos que identificam unicamente o contêiner e o contextualizam. Inclui containerId (o ID longo do contêiner), containerName, image (o ID da imagem), e imageName (o nome e a tag da imagem, ex: fiap-apm-api:latest). Em ambientes Kubernetes, isso é enriquecido com podName, deploymentName, namespace, etc.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

CPU: métricas detalhadas sobre o uso do processador pelo contêiner. O cpuPercent mostra o uso de CPU do contêiner em relação ao total de CPUs do host. O cpuCoresPercent mostra o uso em relação aos cores alocados ao contêiner. Métricas como cpuThrottleTimeMs são cruciais, pois indicam o tempo que o contêiner foi "congelado" (throttled) pelo kernel por ter excedido seus limites de CPU, um sinal claro de subprovisionamento de recursos.

Memória: atributos sobre o consumo de memória. memoryUsageBytes mostra a memória total usada, enquanto memoryCacheBytes e memoryResidentSizeBytes fornecem mais detalhes. O atributo memoryUsageLimitBytes mostra o limite de memória configurado para o contêiner, permitindo calcular facilmente o percentual de uso em relação ao limite.

Rede: métricas de I/O de rede, como networkRxBytes (bytes recebidos), networkTxBytes (bytes transmitidos), e networkRxErrors/networkTxErrors, que ajudam a diagnosticar problemas de conectividade.

Disco: métricas de I/O de disco, como diskReadBytes e diskWriteBytes, que medem o throughput de leitura e escrita do contêiner no sistema de arquivos.

O evento ContainerSample transforma a natureza efêmera e isolada dos contêineres em dados de telemetria estruturados e acionáveis. Ao correlacionar essas métricas com os metadados da aplicação e da orquestração, as equipes de SRE e DevOps ganham a capacidade de gerenciar a performance e a saúde de arquiteturas de microsserviços complexas em escala.

Consultar os dados do ContainerSample via NRQL permite a criação de dashboards que refletem a saúde do ambiente de contêineres, a configuração de alertas precisos sobre o consumo de recursos de um serviço específico e a realização de análises de capacidade para otimizar a alocação de recursos no cluster.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Esta aula abordou os **conceitos fundamentais de monitoramento de infraestrutura**, focando no papel e na arquitetura do **New Relic Infrastructure Agent**. O conteúdo detalhou como ocorre a coleta de métricas essenciais em nível de sistema operacional (hosts) — como CPU, memória, disco, rede e processos — e como esses dados são estruturados e armazenados no evento **SystemSample**.

A análise avançou para o **monitoramento de containers**, explorando sua natureza efêmera e o papel dos **cgroups** no isolamento de recursos. Foi demonstrado como realizar a instrumentação de aplicações .NET em ambientes containerizados, incluindo a instalação do Infrastructure Agent e a coleta de métricas específicas via evento **ContainerSample**. Para organizar esse ecossistema, discutiu-se a definição de identidades lógicas baseada em metadados e tags, garantindo a rastreabilidade dos serviços.

Por fim, o material apresentou o uso da linguagem **NRQL** para consulta, agregação e visualização de dados, fundamentais para a criação de **dashboards de infraestrutura** e a definição de alertas inteligentes. O fechamento da aula focou na correlação estratégica entre métricas de aplicação, container e host, permitindo uma visão holística da saúde do ambiente e uma resolução de problemas mais ágil e precisa.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

REFERÊNCIAS

NEW RELIC. **Infrastructure monitoring overview**. Disponível em: https://docs.newrelic.com/docs/infrastructure/. Acesso em: 28 jan. 2026.

NEW RELIC. **Install the Infrastructure agent.** Disponível em: https://docs.newrelic.com/docs/infrastructure/install-infrastructure-agent/. Acesso em: 28 jan. 2026.

NEW RELIC. **NRQL reference.** Disponível em: https://docs.newrelic.com/docs/nrql/get-started/introduction-nrql-new-relics-query-language/ . Acesso em: 28 jan. 2026.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

PALAVRAS-CHAVE
Infraestrutura, Containers, Métricas.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com



PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com